

Báo cáo Dự án 2: Multi-Agent Pacman

Thông tin sinh viên

Tên thành viên	Vai trò	Phụ trách
Trần Nhật Hưng	Leader	Cài đặt betterEvaluationFunction, rickAwareEvaluationFunction
Vũ Hải Linh	Thành viên	Cài đặt Reflex Agent, Minimax
Phạm Đức Hùng	Thành viên	Cài đặt Expectimax
Nguyễn Duy Đức Minh	Thành viên	Cài đặt Alpha-Beta Pruning

Giới thiệu

Dự án này tập trung vào việc thiết kế và cài đặt các tác nhân (agents) thông minh cho trò chơi Pacman trong môi trường có nhiều đối thủ (multi-agent). Mục tiêu là xây dựng các tác nhân có khả năng đưa ra quyết định tối ưu hoặc gần tối ưu để chiến thắng trò chơi, sử dụng các thuật toán tìm kiếm đối kháng kinh điển trong lĩnh vực Trí tuệ Nhân tạo.

Các thuật toán được cài đặt bao gồm:

- **Reflex Agent:** Tác nhân phản xạ đơn giản.
- **Minimax:** Thuật toán tìm kiếm đối kháng cơ bản.
- **Alpha-Beta Pruning:** Một phiên bản tối ưu của Minimax.
- **Expectimax:** Thuật toán dành cho các đối thủ hoạt động theo xác suất.
- **Evaluation Function Design:** Thiết kế hàm đánh giá trạng thái trò chơi hiệu quả.

1. Tác nhân phản xạ (Reflex Agent)

Tác nhân phản xạ đưa ra quyết định dựa trên việc đánh giá các trạng thái kế tiếp ngay tại thời điểm hiện tại mà không cần nhìn xa hơn.

Hàm đánh giá `evaluationFunction`

Hàm đánh giá được thiết kế để tính điểm cho mỗi hành động tiềm năng của Pacman. Điểm số được tính dựa trên các yếu tố sau:

- **Điểm số cơ bản:** Lấy điểm số của trạng thái kế tiếp làm nền tảng.
- **Khoảng cách đến thức ăn gần nhất:**
 - Khuyến khích Pacman di chuyển đến vị trí gần thức ăn hơn. Điểm thưởng được tính bằng `1.0 / (khoảng cách + 1)`. Điều này giúp Pacman ưu tiên các con đường ngắn nhất để ăn.
- **Tương tác với ma:**
 - **Ma đang sợ hãi (Scared Ghost):** Nếu ma đang trong trạng thái sợ hãi, Pacman được khuyến khích đuổi theo để ăn. Điểm thưởng là `2.0 / (khoảng cách + 1)`.
 - **Ma đang hoạt động (Active Ghost):** Pacman phải tránh xa các con ma đang hoạt động. Một khoản điểm phạt lớn (`-10`) được áp dụng nếu khoảng cách đến ma quá gần (`<= 1`), và một điểm

phạt nhỏ hơn $(-1.5 / (\text{khoảng cách} + 1))$ được áp dụng ở các khoảng cách xa hơn.

- **Lượng thức ăn còn lại:** Một khoản điểm phạt nhỏ $(-0.1 * \text{số lượng thức ăn})$ được thêm vào để khuyến khích Pacman ăn hết thức ăn nhanh chóng.
- **Trạng thái thắng/thua:** Các trạng thái kết thúc game (thắng hoặc thua) được gán giá trị điểm rất lớn hoặc rất nhỏ để đảm bảo tác nhân sẽ chọn nước đi chiến thắng và tránh nước đi dẫn đến thất bại.

```
def evaluationFunction(self, currentGameState: GameState, action):
    # ... (code trích xuất thông tin)
    score = successorGameState.getScore()

    # Ưu tiên di chuyển về phía thức ăn gần nhất
    foodList = newFood.asList()
    if foodList:
        minFoodDist = min(manhattanDistance(newPos, foodPos) for foodPos in
            foodList)
        score += 1.0 / (minFoodDist + 1)

    # Tránh ma đang hoạt động, đuổi ma đang sợ hãi
    for i, ghostState in enumerate(newGhostStates):
        dist = manhattanDistance(newPos, ghostState.getPosition())
        if newScaredTimes[i] > 0:
            score += 2.0 / (dist + 1)
        else:
            if dist <= 1:
                score -= 10
            score -= 1.5 / (dist + 1)

    # Trừ lượng nhỏ điểm cho thức ăn còn lại
    score -= 0.1 * len(foodList)
    return score
```

Kết quả chạy `autograder.py -q q1 --no-graphics`

Starting on 5-11 at 23:51:04

Question q1

=====

```
Pacman emerges victorious! Score: 1429
Pacman emerges victorious! Score: 1190
Pacman emerges victorious! Score: 1245
Pacman emerges victorious! Score: 1237
Pacman emerges victorious! Score: 1254
Pacman emerges victorious! Score: 1248
Pacman emerges victorious! Score: 1431
Pacman emerges victorious! Score: 1252
Pacman emerges victorious! Score: 1257
Pacman emerges victorious! Score: 1249
Average Score: 1279.2
```

```

Scores:      1429.0, 1190.0, 1245.0, 1237.0, 1254.0, 1248.0, 1431.0, 1252.0,
1257.0, 1249.0
Win Rate:    10/10 (1.00)
Record:      Win, Win, Win, Win, Win, Win, Win, Win, Win, Win
*** PASS: test_cases\q1\grade-agent.test (4 of 4 points)
***      1279.2 average score (2 of 2 points)
***      Grading scheme:
***      < 500: 0 points
***      >= 500: 1 points
***      >= 1000: 2 points
***      10 games not timed out (0 of 0 points)
***      Grading scheme:
***      < 10: fail
***      >= 10: 0 points
***      10 wins (2 of 2 points)
***      Grading scheme:
***      < 1: fail
***      >= 1: 0 points
***      >= 5: 1 points
***      >= 10: 2 points

```

Question q1: 4/4

Finished at 23:51:05

Provisional grades

=====

Question q1: 4/4

Total: 4/4

Your grades are NOT yet registered. To register your grades, make sure to follow your instructor's guidelines to receive credit on your project.

2. Thuật toán Minimax

Minimax là thuật toán tìm kiếm đối kháng cho các trò chơi hai người chơi, có tổng bằng không. Trong Pacman, Pacman là tác nhân **MAX** (cố gắng tối đa hóa điểm số), và các con ma là tác nhân **MIN** (cố gắng tối thiểu hóa điểm số của Pacman).

Cách hoạt động

1. Thuật toán xây dựng một cây trò chơi từ trạng thái hiện tại đến một độ sâu (`self.depth`) nhất định.
2. Tại mỗi nút trên cây, thuật toán sẽ:
 - Nếu là lượt của Pacman (nút MAX), nó sẽ chọn hành động dẫn đến trạng thái có giá trị (điểm số) lớn nhất.
 - Nếu là lượt của ma (nút MIN), nó sẽ chọn hành động dẫn đến trạng thái có giá trị nhỏ nhất.
3. Giá trị của các nút lá (ở độ sâu tối đa hoặc trạng thái kết thúc) được tính bằng hàm `self.evaluationFunction`.

4. Giá trị này được truyền ngược lên cây để xác định nước đi tốt nhất cho Pacman ở trạng thái gốc.

```
class MinimaxAgent(MultiAgentSearchAgent):
    def getAction(self, gameState: GameState):
        # ...
        def value(state, depth, agentIndex):
            if depth == self.depth or state.isWin() or state.isLose():
                return self.evaluationFunction(state)
            # ...
            if agentIndex == 0: # Pacman (MAX)
                return max(value(state.generateSuccessor(agentIndex, action),
                                nextDepth, nextAgent) for action in legalActions)
            else: # Ghost (MIN)
                return min(value(state.generateSuccessor(agentIndex, action),
                                nextDepth, nextAgent) for action in legalActions)
        # ...
```

Kết quả chạy `autograder.py -q q2 --no-graphics`

Starting on 5-11 at 23:47:21

Question q2

=====

*** PASS: test_cases\q2\0-eval-function-lose-states-1.test

... (nhiều dòng PASS đã được lược bỏ) ...

*** PASS: test_cases\q2\7-2c-check-depth-two-ghosts.test

*** Running MinimaxAgent on smallClassic 1 time(s).

Pacman died! Score: 84

Average Score: 84.0

Scores: 84.0

Win Rate: 0/1 (0.00)

Record: Loss

*** Finished running MinimaxAgent on smallClassic after 0 seconds.

*** Won 0 out of 1 games. Average score: 84.000000 ***

*** PASS: test_cases\q2\8-pacman-game.test

Question q2: 5/5

Finished at 23:47:21

Provisional grades

=====

Question q2: 5/5

Total: 5/5

3. Cắt tỉa Alpha-Beta (Alpha-Beta Pruning)

Đây là một sự cải tiến của thuật toán Minimax. Nó giúp giảm số lượng nút cần duyệt trên cây trò chơi bằng cách "cắt tỉa" những nhánh mà chắc chắn sẽ không ảnh hưởng đến quyết định cuối cùng.

Cách hoạt động

Thuật toán duy trì hai giá trị trong quá trình duyệt cây:

- **Alpha (α):** Giá trị tốt nhất (lớn nhất) mà tác nhân MAX có thể đảm bảo tại một nút trên đường đi.
- **Beta (β):** Giá trị tốt nhất (nhỏ nhất) mà tác nhân MIN có thể đảm bảo tại một nút trên đường đi.

Quy tắc cắt tỉa:

- Tại một nút MIN, nếu giá trị hiện tại của nó nhỏ hơn hoặc bằng **alpha** của nút cha (nút MAX), nhánh này sẽ bị cắt. Lý do là nút cha MAX sẽ không bao giờ chọn nhánh này vì nó đã có một lựa chọn khác tốt hơn (**alpha**).
- Tại một nút MAX, nếu giá trị hiện tại của nó lớn hơn hoặc bằng **beta** của nút cha (nút MIN), nhánh này sẽ bị cắt. Lý do là nút cha MIN sẽ không bao giờ cho phép đi đến nhánh này vì nó đã có một lựa chọn khác tốt hơn (**beta**).

Việc cắt tỉa này giúp thuật toán tìm kiếm sâu hơn trong cùng một khoảng thời gian, hoặc tìm ra quyết định nhanh hơn ở cùng một độ sâu so với Minimax.

```
class AlphaBetaAgent(MultiAgentSearchAgent):
    def getAction(self, gameState: GameState):
        # ...
        def value(state, depth, agentIndex, alpha, beta):
            # ...
            if agentIndex == 0: # Pacman (MAX)
                v = -float('inf')
                for action in legalActions:
                    v = max(v, value(..., alpha, beta))
                    if v > beta: return v # Cắt tỉa
                    alpha = max(alpha, v)
                return v
            else: # Ghost (MIN)
                v = float('inf')
                for action in legalActions:
                    v = min(v, value(..., alpha, beta))
                    if v < alpha: return v # Cắt tỉa
                    beta = min(beta, v)
                return v
        # ...
```

Kết quả chạy `autograder.py -q q3 --no-graphics`

```

Starting on 5-11 at 23:47:28

Question q3
=====

*** PASS: test_cases\q3\0-eval-function-lose-states-1.test
... (nhiều dòng PASS đã được lược bỏ) ...
*** PASS: test_cases\q3\7-2c-check-depth-two-ghosts.test
*** Running AlphaBetaAgent on smallClassic 1 time(s).
Pacman died! Score: 84
Average Score: 84.0
Scores:      84.0
Win Rate:    0/1 (0.00)
Record:      Loss
*** Finished running AlphaBetaAgent on smallClassic after 0 seconds.
*** Won 0 out of 1 games. Average score: 84.000000 ***
*** PASS: test_cases\q3\8-pacman-game.test

### Question q3: 5/5 ###

Finished at 23:47:29

Provisional grades
=====
Question q3: 5/5
-----
Total: 5/5

```

4. Thuật toán Expectimax

Expectimax được sử dụng khi các đối thủ không hành động một cách tối ưu mà theo một mô hình xác suất. Trong trường hợp này, ta giả định các con ma chọn hành động của chúng một cách ngẫu nhiên và đồng đều từ các nước đi hợp lệ.

Cách hoạt động

- **Nút MAX (Pacman):** Hoạt động giống hệt Minimax, chọn hành động dẫn đến giá trị lớn nhất.
- **Nút CHANCE (Ma):** Thay vì chọn giá trị nhỏ nhất (như nút MIN), nút này tính **giá trị kỳ vọng (expected value)**. Giá trị này là trung bình có trọng số của các giá trị từ các trạng thái kế tiếp. Vì các ma chọn ngẫu nhiên đồng đều, xác suất của mỗi hành động là $1.0 / \text{số lượng hành động hợp lệ}$.

Thuật toán này phù hợp với một mô hình thực tế hơn, nơi các ma không phải lúc nào cũng đưa ra quyết định hoàn hảo để chống lại Pacman.

```

class ExpectimaxAgent(MultiAgentSearchAgent):
    def getAction(self, gameState: GameState):

```

```
# ...
def value(state, depth, agentIndex):
    # ...
    if agentIndex == 0: # Pacman (MAX)
        return max(value(...) for action in legalActions)
    else: # Ghost (CHANCE)
        probability = 1.0 / len(legalActions)
        return sum(probability * value(...) for action in legalActions)
    # ...
```

Kết quả chạy `autograder.py -q q4 --no-graphics`

Starting on 5-11 at 23:47:33

Question q4

=====

*** PASS: test_cases\q4\0-eval-function-lose-states-1.test

... (nhiều dòng PASS đã được lược bỏ) ...

*** PASS: test_cases\q4\6-2c-check-depth-two-ghosts.test

*** Running ExpectimaxAgent on smallClassic 1 time(s).

Pacman died! Score: 84

Average Score: 84.0

Scores: 84.0

Win Rate: 0/1 (0.00)

Record: Loss

*** Finished running ExpectimaxAgent on smallClassic after 0 seconds.

*** Won 0 out of 1 games. Average score: 84.000000 ***

*** PASS: test_cases\q4\7-pacman-game.test

Question q4: 5/5

Finished at 23:47:34

Provisional grades

=====

Question q4: 5/5

Total: 5/5

5. Hàm đánh giá nâng cao (`betterEvaluationFunction`)

Hàm đánh giá này được thiết kế để sử dụng với các thuật toán tìm kiếm đối kháng (`Minimax`, `AlphaBeta`, `Expectimax`) nhằm cung cấp một ước tính chính xác hơn về "chất lượng" của một trạng thái trò chơi.

Các yếu tố đánh giá

- **Điểm số hiện tại:** Là nền tảng của hàm đánh giá.

- **Thức ăn:**
 - **Khoảng cách đến thức ăn gần nhất:** Thường điểm $4.0 / (\text{khoảng cách} + 1)$ để khuyến khích Pacman tích cực tìm ăn.
 - **Số lượng thức ăn còn lại:** Phạt nặng $(-8.0 * \text{số lượng})$ để thúc đẩy việc dọn sạch bản đồ.
 - **Bonus khi hết thức ăn:** Thưởng một lượng điểm lớn $(+1000)$ khi ăn hết thức ăn.
- **Viên năng lượng (Capsules):**
 - Tương tự như thức ăn, Pacman được khuyến khích ăn viên năng lượng với điểm thưởng $4.0 / (\text{khoảng cách} + 1)$.
 - Phạt rất nặng $(-50.0 * \text{số lượng})$ nếu còn nhiều viên năng lượng, vì chúng rất quan trọng trong chiến thuật.
- **Ma:**
 - **Ma sợ hãi:** Ưu tiên rất cao cho việc săn ma sợ hãi, với điểm thưởng $10.0 / (\text{khoảng cách} + 1)$.
 - **Ma hoạt động:** Xây dựng một "vùng nguy hiểm" xung quanh ma:
 - Phạt cực nặng (-500) nếu khoảng cách là 1.
 - Phạt nặng (-100) nếu khoảng cách là 2.
 - Phạt vừa phải (-25) nếu khoảng cách là 3.
 - Phạt nhẹ ở các khoảng cách xa hơn. Điều này tạo ra một gradient giúp Pacman tránh xa ma một cách mượt mà.

Sự kết hợp của các yếu tố này tạo ra một hành vi phức tạp và hiệu quả cho Pacman: vừa tích cực ăn điểm, vừa biết cách săn mồi và lẩn tránh kẻ thù một cách thông minh.

Kết quả chạy `autograder.py -q q5 --no-graphics`

```
Starting on 5-11 at 23:47:38
```

```
Question q5
```

```
=====
```

```
Pacman emerges victorious! Score: 1352
Pacman emerges victorious! Score: 1335
Pacman emerges victorious! Score: 1160
Pacman emerges victorious! Score: 1142
Pacman emerges victorious! Score: 1349
Pacman emerges victorious! Score: 1371
Pacman emerges victorious! Score: 1373
Pacman emerges victorious! Score: 1366
Pacman emerges victorious! Score: 1151
Pacman emerges victorious! Score: 1357
Average Score: 1295.6
Scores:      1352.0, 1335.0, 1160.0, 1142.0, 1349.0, 1371.0, 1373.0, 1366.0,
1151.0, 1357.0
Win Rate:    10/10 (1.00)
Record:      Win, Win, Win, Win, Win, Win, Win, Win, Win, Win
*** PASS: test_cases\q5\grade-agent.test (6 of 6 points)
***      1295.6 average score (2 of 2 points)
***      Grading scheme:
***      < 500:  0 points
***      >= 500: 1 points
```



```

***      >= 1000:  2 points
***      10 games not timed out (1 of 1 points)
***      Grading scheme:
***      < 0:  fail
***      >= 0:  0 points
***      >= 10:  1 points
***      10 wins (3 of 3 points)
***      Grading scheme:
***      < 1:  fail
***      >= 1:  1 points
***      >= 5:  2 points
***      >= 10: 3 points

```

Question q5: 6/6

Finished at 23:47:41

Provisional grades

=====

Question q5: 6/6

Total: 6/6

6. Hàm đánh giá nhận biết rủi ro (**riskAwareEvaluationFunction**)

Đây là một phiên bản nâng cao hơn của hàm đánh giá, tập trung vào việc tính toán **khoảng cách thực tế trong mê cung (maze distance)** thay vì khoảng cách Manhattan. Điều này mang lại một sự hiểu biết chính xác hơn về bản đồ và các mối đe dọa.

Các cải tiến chính

1. Sử dụng Khoảng cách Mê cung (Maze Distance):

- Thay vì dùng **manhattanDistance** (đường chim bay), hàm này sử dụng thuật toán **Tìm kiếm theo chiều rộng (BFS)** để tính toán khoảng cách ngắn nhất thực tế giữa hai điểm, có tính đến các bức tường. Điều này giúp Pacman không bị "lừa" bởi những con ma ở gần nhưng thực tế lại bị ngăn cách bởi tường.
- Một **distMap** được tạo ra từ vị trí của Pacman đến mọi điểm khác trên bản đồ, giúp tối ưu hóa việc truy vấn khoảng cách.

2. Đánh giá Rủi ro từ Ma một cách Tinh vi hơn:

- Áp lực từ nhiều ma:** Hàm tính toán điểm phạt dựa trên khoảng cách đến *tất cả* các ma đang hoạt động, không chỉ con ma gần nhất. Điều này giúp Pacman nhận biết được tình huống bị bao vây.
- Phạt nặng khi ở gần:** Nếu khoảng cách đến ma đang hoạt động nhỏ hơn hoặc bằng 2, một khoản điểm phạt rất lớn được áp dụng ($-60.0 * (3 - \text{dist})$), khiến Pacman phải ưu tiên việc thoát khỏi nguy hiểm ngay lập tức.

- **Ưu tiên sinh tồn:** Một điểm phạt bổ sung $(-12.0 / (\text{minActiveDist} + 0.1))$ được áp dụng dựa trên khoảng cách đến con ma gần nhất, nhấn mạnh rằng việc sống sót là ưu tiên hàng đầu.

3. Nhận biết Ngõ cụt (Mobility Awareness):

- Hàm đánh giá "sự linh hoạt" (mobility) của Pacman bằng cách đếm số lượng nước đi hợp lệ (không tính **STOP**).
- Nếu Pacman chỉ có 1 hoặc 2 nước đi (dấu hiệu của ngõ cụt hoặc hành lang hẹp), nó sẽ bị trừ điểm.
- Đặc biệt, nếu đang bị ma truy đuổi ở khoảng cách gần và đồng thời đang ở trong một khu vực kém linh hoạt, điểm phạt sẽ rất nặng, buộc Pacman phải tìm đường thoát.

So sánh với `betterEvaluationFunction`

- **Độ chính xác:** `riskAwareEvaluationFunction` chính xác hơn nhiều trong việc đánh giá khoảng cách và rủi ro thực tế.
- **Chi phí tính toán:** Việc chạy BFS để tính `mazeDistance` ở mỗi trạng thái làm cho hàm này tốn nhiều tài nguyên tính toán hơn so với việc dùng `manhattanDistance`.
- **Hành vi:** Pacman với hàm này có xu hướng chơi "an toàn" hơn, cẩn trọng hơn khi di chuyển vào các khu vực hẹp và rất nhạy cảm với vị trí của ma.

```
def riskAwareEvaluationFunction(currentGameState: GameState):
    # ... (BFS để tính maze distance) ...

    # Tìm khoảng cách thực tế đến thức ăn gần nhất
    minFoodDist = minMazeDistance(foodList)
    if minFoodDist is not None:
        score += 3.0 / (minFoodDist + 1)
        score -= 4.0 * len(foodList)

    # ... (Đánh giá khoảng cách đến viên năng lượng) ...

    # Đánh giá mối đe dọa từ ma
    activeGhostDists = []
    for ghostState in ghostStates:
        dist = mazeDistance(ghostState.getPosition())
        # ...
        if ghostState.scaredTimer > 0:
            score += 8.0 / (dist + 1)
        else:
            activeGhostDists.append(dist)
            score -= 3.0 / (dist + 0.5)
            if dist <= 2:
                score -= 60.0 * (3 - dist)

    # Phạt dựa trên con ma gần nhất
    if activeGhostDists:
        minActiveDist = min(activeGhostDists)
        score -= 12.0 / (minActiveDist + 0.1)
```

```
# Phạt khi vào ngõ cụt
mobility = len([action for action in currentGameState.getLegalActions(0) if
action != Directions.STOP])
if mobility <= 1:
    score -= 25.0
# ...
return score
```
